

环境：

- 关掉 conda，用 `uv` 管理项目 `mor26-sheet0`。
- JAX 装在此项目 `.venv` 里，一般一个项目文件下只能有一个环境
- 本地运行，默认使用 CPU

```
In [22]: import numpy as np
import jax
import jax.numpy as jnp
import time
import matplotlib.pyplot as plt
```

Exercise 0.1: uv 与 JAX

在你的笔记本电脑上安装 Python 包管理器 `uv` (<https://docs.astral.sh/uv/>)。

(a) 用 `uv init mor26-sheet0` 创建一个新项目。用 `uv add jax` 添加 JAX。

(b) 编写一个脚本 `hello_jax.py`，打印 `jax.devices()`，采样一个矩阵 $A \in \mathbb{R}^{4 \times 4}$ ，其元素为独立标准正态分布，并打印 $A^T A$ 。用 `uv run hello_jax.py` 运行它。
JAX 默认使用什么硬件？

```
In [2]: # 打印 JAX 设备
print("JAX devices:", jax.devices())

# 采样 4x4 矩阵，元素独立标准正态
key = jax.random.PRNGKey(0)
A = jax.random.normal(key, (4, 4))
print("A =\n", A)
print("A^T A =\n", A.T @ A)
```

```
JAX devices: [CpuDevice(id=0)]
A =
[[ 0.08482574  1.9097648  0.29561743  1.120948 ]
 [ 0.33432344 -0.82606775  0.6481277  1.0434873 ]
 [-0.7824839  -0.4539462  0.6297971  0.81524646]
 [-0.32787678 -1.1234448  -1.6607416  0.27290547]]
A^T A =
[[ 0.8387518  0.6093805  0.29347277 -0.28344905]
 [ 0.6093805  5.797785  1.6090199  0.6020833 ]
 [ 0.29347277  1.6090199  3.662166  1.0678991 ]
 [-0.28344905  0.6020833  1.0678991  3.0844944 ]]
```

Exercise 0.2: 行交换, NumPy 与 JAX 对比

考虑以下 Python 脚本，它交换矩阵 A 的第 i 行和第 j 行，例如部分主元法需要这样做。

- (a) 将该函数应用于一个矩阵 $A \in \mathbb{R}^{4 \times 4}$ 。描述它对实参的影响。
- (b) 将同一函数应用于同一个矩阵的 `jax.numpy` (`jnp`) 数组。会发生什么？
- (c) 编写一个与 JAX 兼容的函数 `swap_rows_jax(A, i, j)`，返回一个交换了第 i 行和第 j 行的矩阵。

```
In [14]: # 用 numpy 生成 A_np 的时候, 使用函数会改变 A_np(以往学习的常识)
def swap_rows(A, i, j):
    A[[i, j]] = A[[j, i]]
    return A

A_np = np.arange(1, 17).reshape(4, 4).astype(float)
print("Origin:\n", A_np)

B_np = swap_rows(A_np, 0, 2)
print("Returned:\n", B_np)
print("After call the function:\n", A_np)
print("Same object?", B_np is A_np)
```

```
Origin:
[[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]
[13. 14. 15. 16.]]
Returned:
[[ 9. 10. 11. 12.]
 [ 5.  6.  7.  8.]
 [ 1.  2.  3.  4.]
[13. 14. 15. 16.]]
After call the function:
[[ 9. 10. 11. 12.]
 [ 5.  6.  7.  8.]
 [ 1.  2.  3.  4.]
[13. 14. 15. 16.]]
Same object? True
```

```
In [11]: # 使用 jax 生成 A_jnp 的时候, jnp 生成的数组是不可以改变的 只能先取copy
A_jnp = jnp.arange(1, 17).reshape(4, 4).astype(float)
print("Before:\n", A_jnp)

try:
    B_jnp = swap_rows(A_jnp, 0, 2)
    print("Returned:\n", B_jnp)
    print("Original after call:\n", A_jnp)
except Exception as e:
    print("Error:", type(e).__name__)
    print(e)
```

```
Before:
[[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]
[13. 14. 15. 16.]]
Error: TypeError
Using a non-tuple sequence for multidimensional indexing is not allowed; use `arr
[array(seq)]` instead of `arr[seq]`. See https://github.com/google/jax/issues/456
4 for more information.
```

```
In [13]: # 意思是因为 jnp 序列无法更改, 所以这里用 index 创建了一个新序列去做修改, 而不是在
def swap_rows_jax(A, i, j):
    idx = jnp.array([i, j])
    return A.at[idx].set(A[idx[::-1]])

A_jnp2 = jnp.arange(1, 17).reshape(4, 4).astype(float)
print("Before:\n", A_jnp2)
```

```
B_jnp2 = swap_rows_jax(A_jnp2, 0, 2)
print("Returned:\n", B_jnp2)
print("Original after call:\n", A_jnp2)
```

Before:

```
[[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]
 [13. 14. 15. 16.]]
```

Returned:

```
[[ 9. 10. 11. 12.]
 [ 5.  6.  7.  8.]
 [ 1.  2.  3.  4.]
 [13. 14. 15. 16.]]
```

Original after call:

```
[[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]
 [13. 14. 15. 16.]]
```

Exercise 0.3: 二次函数的梯度

设 $A = A^T \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, 设

$$f(x) := \frac{1}{2} x^T A x - b^T x.$$

- (a) 编写一个与 JAX 兼容的函数 `f(A, b, x)`, 用于计算 $f(x)$ 。
- (b) 手工计算 $\nabla f(x)$, 并编写一个函数 `grad_f(A, b, x)`, 返回 $\nabla f(x)$ 。
- (c) 编写一个函数 `jax_grad_f(A, b, x)`, 使用 `jax.grad` 返回梯度 $\nabla f(x)$ 。

```
In [15]: def f(A, b, x):
          return 0.5 * x.T @ A @ x - b.T @ x

          def grad_f(A, b, x):
              return A @ x - b

          def jax_grad_f(A, b, x):
              return jax.grad(f, argnums=2)(A, b, x)
```

```
In [16]: key = jax.random.PRNGKey(42)
          k1, k2, k3 = jax.random.split(key, 3) # 随机种子

          n = 5
          R = jax.random.normal(k1, (n, n))
          A_sym = R + R.T
          b_vec = jax.random.normal(k2, (n,))
          x_vec = jax.random.normal(k3, (n,))

          g_manual = grad_f(A_sym, b_vec, x_vec)
          g_auto = jax_grad_f(A_sym, b_vec, x_vec)

          print("Manual gradient:\n", g_manual)
          print("JAX gradient:\n", g_auto)
          print("Difference norm:", jnp.linalg.norm(g_manual - g_auto))
```

Manual gradient:

```
[ 0.8425228  5.9060946  3.0302925  3.2551088 -3.3287349]
```

JAX gradient:

```
[ 0.84252274  5.9060946  3.0302925  3.2551088 -3.3287349 ]
```

Difference norm: 5.9604645e-08

Exercise 0.4: 参数化求解、快照与计时

设 $A_0, A_1 \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, 并且对于参数 $\mu \in \mathbb{R}$, 设

$$A(\mu) := A_0 + \mu A_1, \quad u(\mu) := A(\mu)^{-1}b.$$

(a) 编写一个函数 `solve(mu, A0, A1, b)`, 使用 `jnp.linalg.solve` 返回 $u(\mu)$ 。

(b) 编写一个函数 `snapshots(mus, A0, A1, b)`, 对于以向量 `mus` 给出的参数 $\mu_1, \dots, \mu_k$, 返回快照矩阵 $U := [u(\mu_1) \dots u(\mu_k)] \in \mathbb{R}^{n \times k}$, 并使用一次 `jax.vmap` 调用。不要对参数进行循环。

(d) 打开 <https://colab.research.google.com/>, 切换到 GPU 运行时, 并用

`jax.devices()` 检查 JAX 是否使用 GPU。对不断增大的 n 重复 (c)。对于哪些问题规模, GPU 才有优势?

```
In [17]: # (a) 和 (b)
def solve(mu, A0, A1, b):
    return jnp.linalg.solve(A0 + mu * A1, b)

def snapshots(mus, A0, A1, b):
    return jax.vmap(solve, in_axes=(0, None, None, None))(mus, A0, A1, b)
```

(c) 生成随机测试数据, 其中 $n = 1024$, $k = 16$, $A_0 = nI + R + R^T$, R 为标准正态分布, A_1 和 b 也为标准正态分布, $\mu_1, \dots, \mu_k \in [0, 1]$ 等距分布。使用测试数据比较以下三种方式的运行时间: 对 `solve` 的 Python 循环、`snapshots`、以及 `jax.jit(snapshots)`; 单独对编译函数的第一次调用计时, 并使用 `jax.block_until_ready`。哪种变体最快?

```
In [19]: n = 1024; k = 16

key = jax.random.PRNGKey(0)
k1, k2, k3 = jax.random.split(key, 3)

R = jax.random.normal(k1, (n, n))
A0 = n * jnp.eye(n) + R + R.T
A1 = jax.random.normal(k2, (n, n))
b = jax.random.normal(k3, (n,))
mus = jnp.linspace(0.0, 1.0, k)

print("Shapes:", A0.shape, A1.shape, b.shape, mus.shape)
```

Shapes: (1024, 1024) (1024, 1024) (1024,) (16,)

```
In [20]: # 1. Python 循环
def loop_solve(mus, A0, A1, b):
    return jnp.stack([solve(mu, A0, A1, b) for mu in mus])

t0 = time.time()
U_loop = loop_solve(mus, A0, A1, b)
U_loop.block_until_ready()
```

```

t1 = time.time()
print(f"Python loop:           {t1 - t0:.4f} s")

# 2. vmap, 未 JIT
t0 = time.time()
U_vmap = snapshots(mus, A0, A1, b)
U_vmap.block_until_ready()
t1 = time.time()
print(f"vmap (no jit):         {t1 - t0:.4f} s")

# 3. jit(vmap)
snapshots_jit = jax.jit(snapshots)

# 第一次调用, 含编译
t0 = time.time()
U_jit = snapshots_jit(mus, A0, A1, b)
U_jit.block_until_ready()
t1 = time.time()
print(f"jit(vmap) first call:  {t1 - t0:.4f} s")

# 第二次调用, 已编译
t0 = time.time()
U_jit = snapshots_jit(mus, A0, A1, b)
U_jit.block_until_ready()
t1 = time.time()
print(f"jit(vmap) second call: {t1 - t0:.4f} s")

```

```

Python loop:           1.3040 s
vmap (no jit):         1.6220 s
jit(vmap) first call:  1.3760 s
jit(vmap) second call: 0.6260 s

```

(e) 用 `jnp.linalg.svd` 计算快照矩阵的奇异值分解 $U = W\Sigma V^T$, 并在半对数坐标上绘制奇异值 $\sigma_1 \geq \dots \geq \sigma_k$. 确定最小的 r , 使得 U 的最佳秩 r 近似 U_r 满足 $\|U - U_r\|_F \leq 10^{-5} \|U\|_F$. 使用 `jnp.einsum` 计算 Frobenius 范数。

```

In [23]: # SVD 和 Frobenius 范数

U = U_jit # 用前面 jit(vmap) 算出的 snapshot 矩阵
W, S, Vt = jnp.linalg.svd(U, full_matrices=False)
print("Singular values:\n", S)

def frob_norm(M):
    return jnp.sqrt(jnp.einsum('ij,ij->', M, M))

U_norm = frob_norm(U)
print("||U||_F =", U_norm)

```

```

Singular values:
[1.2636644e-01 1.2083595e-03 1.0456248e-05 8.7444036e-08 3.3526163e-08
 5.9800209e-09 5.7252967e-09 5.3532312e-09 5.1401581e-09 5.0317364e-09
 4.8808917e-09 4.6749471e-09 4.5913491e-09 4.4832658e-09 4.2506949e-09
 4.1308046e-09]
||U||_F = 0.12637226

```

```

In [24]: # 遍历 r 找最小误差

errors = []
for r in range(1, k + 1):
    Ur = (W[:, :r] * S[:r]) @ Vt[:r, :]

```

```

err = frob_norm(U - Ur)
errors.append(err)
print(f"r = {r:2d}, ||U - Ur||_F = {err:.3e}, ratio = {err / U_norm:.3e}")

errors = jnp.array(errors)
mask = errors <= 1e-5 * U_norm
if bool(mask.any()):
    r_min = int(jnp.argmax(mask)) + 1
    print("Smallest r with ratio <= 1e-5:", r_min)
else:
    print("No r <= k satisfies the tolerance.")

```

```

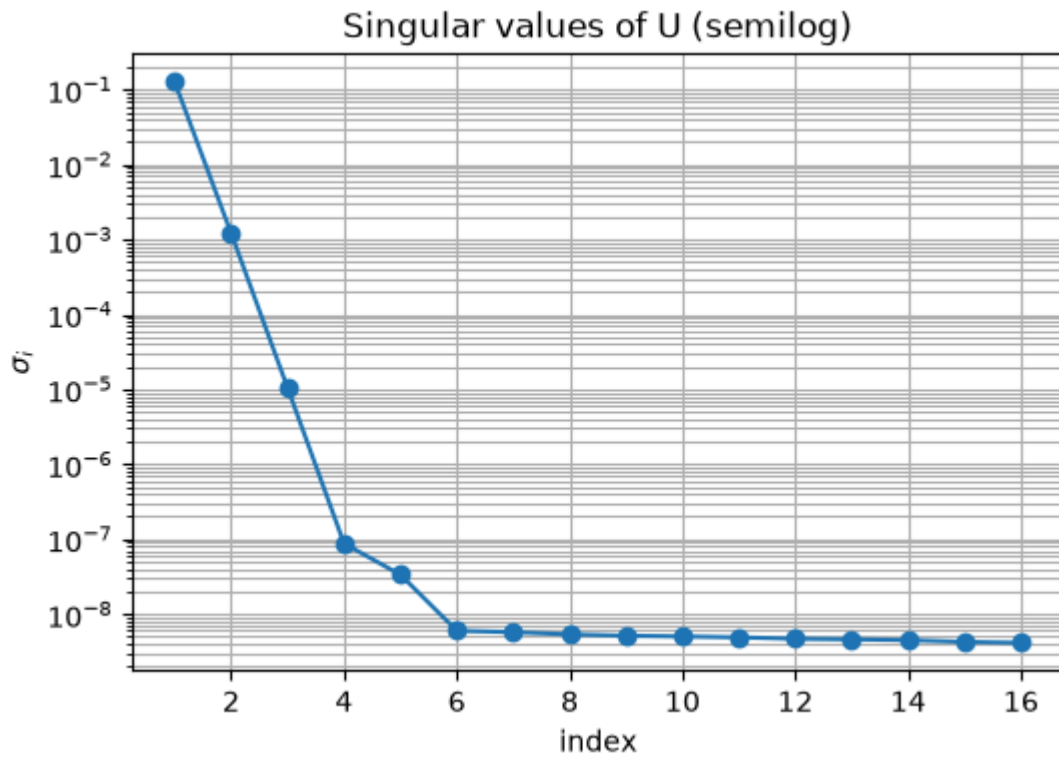
r = 1, ||U - Ur||_F = 1.208e-03, ratio = 9.562e-03
r = 2, ||U - Ur||_F = 1.046e-05, ratio = 8.274e-05
r = 3, ||U - Ur||_F = 1.120e-07, ratio = 8.865e-07
r = 4, ||U - Ur||_F = 7.080e-08, ratio = 5.603e-07
r = 5, ||U - Ur||_F = 7.802e-08, ratio = 6.174e-07
r = 6, ||U - Ur||_F = 7.785e-08, ratio = 6.161e-07
r = 7, ||U - Ur||_F = 7.771e-08, ratio = 6.149e-07
r = 8, ||U - Ur||_F = 7.758e-08, ratio = 6.139e-07
r = 9, ||U - Ur||_F = 7.753e-08, ratio = 6.135e-07
r = 10, ||U - Ur||_F = 7.742e-08, ratio = 6.126e-07
r = 11, ||U - Ur||_F = 7.729e-08, ratio = 6.116e-07
r = 12, ||U - Ur||_F = 7.719e-08, ratio = 6.108e-07
r = 13, ||U - Ur||_F = 7.709e-08, ratio = 6.100e-07
r = 14, ||U - Ur||_F = 7.701e-08, ratio = 6.094e-07
r = 15, ||U - Ur||_F = 7.691e-08, ratio = 6.086e-07
r = 16, ||U - Ur||_F = 7.683e-08, ratio = 6.080e-07
Smallest r with ratio <= 1e-5: 3

```

```

In [25]: # 奇异值分解的绘图
plt.figure(figsize=(6, 4))
plt.semilogy(range(1, k + 1), S, 'o-')
plt.xlabel("index")
plt.ylabel(r"$\sigma_i$")
plt.title("Singular values of U (semilog)")
plt.grid(True, which="both")
plt.show()

```



Exercise 0.5: EPFL 集群: SCITAS 与 Izar

- (a) 请 A. Karsai 将你加入 SCITAS 用户组, 以获得 EPFL HPC 集群的访问权限。
(b) 使用以下命令测试连接

```
ssh <username>@izar.hpc.epfl.ch
```